
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 5

Funções em C

Exercícios (5.8–5.48)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 5.8 – Valores de x	2
2	Exercício 5.9 – Tarifas de Estacionamento	2
3	Exercício 5.10 – Arredondando para Inteiro	4
4	Exercício 5.11 – Arredondamento por Casa Decimal	4
5	Exercício 5.12 – Perguntas sobre Aleatoriedade	6
6	Exercício 5.13 – Geração de aleatórios em intervalos	6
7	Exercício 5.14 – Aleatórios de conjuntos específicos	7
8	Exercício 5.15 – Cálculo de Hipotenusa	8
9	Exercício 5.16 – Função potenciaInt	9
10	Exercício 5.17 – Função multiple	10
11	Exercício 5.18 – Função even	11
12	Exercício 5.19 – Quadrado Sólido de Asteriscos	11
13	Exercício 5.20 – Quadrado de Caractere Variável	12
14	Exercício 5.21 – Projeto: Desenhando Formas	14
15	Exercício 5.22 – Separando Dígitos	16
16	Exercício 5.23 – Tempo em Segundos	18
17	Exercício 5.24 – Conversões de Temperatura	18
18	Exercício 5.25 – Mínimo de 3 Números	19
19	Exercício 5.26 – Números Perfeitos	21
20	Exercício 5.27 – Números Primos	22
21	Exercício 5.28 – Inverter Dígitos	24
22	Exercício 5.29 – MDC (Algoritmo de Euclides)	24
23	Exercício 5.30 – Pontos de Qualidade	25
24	Exercício 5.31 – Lançamento de Moeda	26
25	Exercício 5.32 – Descubra o Número	27
26	Exercício 5.33 – Modificação de “Descubra o Número”	28

27 Exercício 5.34 – Exponenciação Recursiva	30
28 Exercício 5.35 – Fibonacci Iterativo com double	30
29 Exercício 5.36 – Torres de Hanói	32
30 Exercício 5.38 – Visualizando a Recursão (Fatorial)	33
31 Exercício 5.39 – MDC Recursivo	35
32 Exercício 5.40 – main Recursiva	35
33 Exercício 5.41 – Distância entre Pontos	36
34 Exercício 5.42 – O que o programa faz?	37
35 Exercício 5.43 – O que faz mystery?	37
36 Exercício 5.44 – mystery sem restrição em b	37
37 Exercício 5.46 – Identificar e Corrigir Erros	39
38 Exercício 5.47 – Modificação do Jogo Craps	40
39 Exercício 5.48 – Aperfeiçoando Fibonacci Recursivo	43

1. Exercício 5.8 – Valores de x

Resposta detalhada

- a) `fabs(7.5) → |7,5| = 7,5`
- b) `floor(7.5) → maior inteiro ≤ 7,5 = 7,0`
- c) `fabs(0.0) → 0,0`
- d) `ceil(0.0) → 0,0`
- e) `fabs(-6.4) → 6,4`
- f) `ceil(-6.4) → menor inteiro ≥ -6,4 = -6,0`
- g) `ceil( -fabs( -8 + floor(-5.5) ) )`

Avaliando de dentro para fora:

- `floor(-5.5) = -6`
- `-8 + (-6) = -14`
- `fabs(-14) = 14`
- `-(14) = -14`
- `ceil(-14) = -14,0`

Boa prática de programação

Atenção a `floor` de números negativos: `floor(-5.5)` é -6 (mais negativo), não -5. Já `ceil(-5.5)` é -5. Pense em *piso* como “arredondar para baixo na reta numérica”.

2. Exercício 5.9 – Tarifas de Estacionamento

Resposta detalhada

Regras:

- Tarifa mínima: R\$ 2,00 (até 3 horas)
- Acima de 3 horas: R\$ 0,50 adicional por hora ou fração
- Máximo: R\$ 10,00 em qualquer período de 24 horas

```
1  /* Ex5.9: tarifa_estacionamento.c
2     Calcula taxas de estacionamento para 3 clientes */
3  #include <stdio.h>
4  #include <math.h>
5
6  double calcularTaxas( double horas ); /* prototipo */
7
8  int main( void )
9  {
10     double horas;           /* horas estacionado */
11     double taxa;            /* tarifa para o cliente
12                               atual */
12     double totalHoras = 0.0;
13     double totalReceita = 0.0;
14     int cliente;
```

```
15
16     printf( "%5s%8s%10s\n", "Carro", "Horas", "Taxa" )
17     ;
18     for ( cliente = 1; cliente <= 3; ++cliente ) {
19         printf( "Digite horas do carro %d: ", cliente
20             );
21         scanf( "%lf", &horas );
22
23         taxa = calcularTaxas( horas );
24
25         printf( "%5d%8.1f%10.2f\n", cliente, horas,
26             taxa );
27         totalHoras += horas;
28         totalReceita += taxa;
29     }
30
31     printf( "%5s%8.1f%10.2f\n", "TOTAL", totalHoras,
32         totalReceita );
33     return 0;
34 } /* fim da funcao main */
35
36 /* Calcula a tarifa segundo as regras */
37 double calcularTaxas( double horas )
38 {
39     double taxa = 2.00; /* taxa minima ate 3 horas */
40
41     if ( horas > 3.0 ) {
42         /* Cobra 0,50 por hora ou fracao acima de 3
43             horas */
44         /* ceil arredonda fracao para a hora cheia
45             seguinte */
46         taxa += 0.50 * ceil( horas - 3.0 );
47     }
48
49     if ( taxa > 10.00 ) {
50         taxa = 10.00; /* limite maximo diario */
51     }
52
53     return taxa;
54 } /* fim da funcao calcularTaxas */
```

Análise: Para 1,5 horas, está dentro das 3h iniciais → R\$ 2,00. Para 4 horas, $\text{ceil}(4-3) = 1$ hora extra → $2 + 0,50 = \text{R\$ } 2,50$. Para 24 horas, calculado seria $2 + 0,50 \times 21 = 12,50$, mas limita-se em R\$ 10,00.

3. Exercício 5.10 – Arredondando para Inteiro

Resposta detalhada

Truque: `floor(x + 0.5)` arredonda `x` para o inteiro mais próximo. Por quê? Se a parte fracionária for $\geq 0,5$, somar 0,5 “empurra” o número para o próximo inteiro.

```
1  /* Ex5.10: arredonda.c */
2  #include <stdio.h>
3  #include <math.h>
4
5  int main( void )
6  {
7      double x, arredondado;
8
9      printf( "Digite numero (negativo para terminar): "
10             );
11      scanf( "%lf", &x );
12
13      while ( x >= 0.0 ) {
14          arredondado = floor( x + 0.5 );
15          printf( "Original: %.4f -> Arredondado: %.0f\n",
16                  x, arredondado );
17          printf( "Proximo (negativo para terminar): " );
18          ;
19          scanf( "%lf", &x );
20      }
21
22      return 0;
23 }
```

4. Exercício 5.11 – Arredondamento por Casa Decimal

Resposta detalhada

```
1  /* Ex5.11: arredonda_casas.c */
2  #include <stdio.h>
3  #include <math.h>
4
5  double arredInteiro      ( double x );
6  double arredDecimos     ( double x );
7  double arredCentesimos  ( double x );
8  double arredMilesimos   ( double x );
9
10 int main( void )
11 {
```

```
12     double x;
13     printf( "Digite um numero: " );
14     scanf( "%lf", &x );
15
16     printf( "Original:      %.6f\n", x );
17     printf( "Inteiro:       %.0f\n", arredInteiro( x ) )
18     ;
19     printf( "Decimos:      %.1f\n", arredDecimos( x ) )
20     ;
21     printf( "Centesimos:   %.2f\n", arredCentesimos( x
22     ) );
23     printf( "Milesimos:    %.3f\n", arredMilesimos( x )
24     );
25
26     return 0;
27 }
28
29 double arredInteiro( double x )
30 { return floor( x + 0.5 ); }
31
32 double arredDecimos( double x )
33 { return floor( x * 10.0 + 0.5 ) / 10.0; }
34
35 double arredCentesimos( double x )
36 { return floor( x * 100.0 + 0.5 ) / 100.0; }
37
38 double arredMilesimos( double x )
39 { return floor( x * 1000.0 + 0.5 ) / 1000.0; }
```

Lógica: para arredondar a n casas decimais, multiplique por 10^n , arredonde para inteiro, e divida de volta por 10^n . Por exemplo, para arredondar 3,14159 a 2 casas: $3,14159 \times 100 = 314,159$; $\text{floor}(314,659) = 314$; $314/100 = 3,14$.

5. Exercício 5.12 – Perguntas sobre Aleatoriedade

Resposta detalhada

- a) **Escolher números “aleatoriamente”** significa selecioná-los de tal modo que cada valor possível tenha igual probabilidade de ser escolhido (distribuição uniforme), e que escolhas sucessivas sejam estatisticamente independentes umas das outras. Computadores não geram aleatoriedade verdadeira – usam algoritmos determinísticos para produzir sequências que *parecem* aleatórias (pseudoaleatoriedade).
- b) **rand** é útil para simular jogos de azar porque modela situações como lançamento de dados, baralho, moeda – em que o resultado depende do acaso. Sem **rand**, todas as execuções produziriam o mesmo resultado, perdendo a imprevisibilidade característica do jogo.
- c) **Por que usar srand?** Sem **srand**, o gerador **rand** produz sempre a **mesma sequência** a cada execução do programa. **srand(time(NULL))** fornece uma semente diferente a cada vez (baseada no relógio do sistema), garantindo sequências distintas.

Quando não randomizar? Durante o desenvolvimento e depuração, queremos resultados *reproduzíveis* – o programa deve se comportar exatamente igual a cada execução. Nesse contexto, omitir **srand** (ou usar uma semente fixa) é desejável para identificar e corrigir bugs.

- d) **Escala e deslocamento:** **rand()** retorna valores entre 0 e **RAND_MAX**. Para gerar valores em outro intervalo $[a, b]$:

- **Escala:** restringir a faixa a $b - a + 1$ valores: **rand() % (b - a + 1)**
- **Deslocamento:** mover para começar em a : $a + \dots$

A fórmula completa: $a + \text{rand()} \% (b - a + 1)$.

- e) **Por que simulações são úteis?** Permitem investigar fenômenos do mundo real:

- Quando experimentos reais são caros (testes de carros, aviões)
- Perigosos (reações nucleares, epidemiologia)
- Lentos (evolução, mudança climática)
- Ou impossíveis (cenários “e se?”)

A simulação permite testar hipóteses, prever resultados e tomar decisões informadas em segundos, com custo mínimo.

6. Exercício 5.13 – Geração de aleatórios em intervalos

Resposta detalhada

Fórmula geral: $n = a + \text{rand()} \% (b - a + 1)$, gerando valores em $[a, b]$.

- a) $1 \leq n \leq 2$: $n = 1 + \text{rand()} \% 2$;
b) $1 \leq n \leq 100$: $n = 1 + \text{rand()} \% 100$;
c) $0 \leq n \leq 9$: $n = \text{rand()} \% 10$;
d) $1000 \leq n \leq 1112$: $n = 1000 + \text{rand()} \% 113$; $(1112 - 1000 + 1 = 113)$
e) $-1 \leq n \leq 1$: $n = -1 + \text{rand()} \% 3$; $(1 - (-1) + 1 = 3)$

f) $-3 \leq n \leq 11$: $n = -3 + \text{rand}() \% 15$; $(11 - (-3) + 1 = 15)$

7. Exercício 5.14 – Aleatórios de conjuntos específicos

Resposta detalhada

Estratégia: cada conjunto é uma progressão aritmética (PA) com 5 termos. Geramos um inteiro aleatório no intervalo apropriado e aplicamos a fórmula $\text{primeiro_termo} + i \cdot \text{razão}$.

a) **2, 4, 6, 8, 10:** primeiro 2, razão 2, 5 termos:

```
1 printf( "%d", 2 + 2 * ( rand() % 5 ) );
```

b) **3, 5, 7, 9, 11:** primeiro 3, razão 2, 5 termos:

```
1 printf( "%d", 3 + 2 * ( rand() % 5 ) );
```

c) **6, 10, 14, 18, 22:** primeiro 6, razão 4, 5 termos:

```
1 printf( "%d", 6 + 4 * ( rand() % 5 ) );
```

8. Exercício 5.15 – Cálculo de Hipotenusa

Resposta detalhada

```
1  /* Ex5.15: hipotenusa.c
2     Calcula hipotenusa de triangulo retangulo */
3  #include <stdio.h>
4  #include <math.h>
5
6  double hipotenusa( double cateto1, double cateto2 );
7
8  int main( void )
9  {
10     double s1, s2;
11     int triangulo;
12
13     /* Tabela 5.18: 3 triangulos a testar */
14     double lado1[] = { 3.0, 5.0, 8.0 };
15     double lado2[] = { 4.0, 12.0, 15.0 };
16
17     printf( "%-10s%-10s%-10s%-15s\n",
18             "Triangulo", "Lado 1", "Lado 2", "
19             Hipotenusa" );
20
21     for ( triangulo = 0; triangulo < 3; ++triangulo )
22     {
23         s1 = lado1[ triangulo ];
24         s2 = lado2[ triangulo ];
25         printf( "%-10d%-10.1f%-10.1f%-15.2f\n",
26                 triangulo + 1, s1, s2, hipotenusa( s1,
27                 s2 ) );
28     }
29
30     return 0;
31 }
32
33 /* Calcula hipotenusa: sqrt(a^2 + b^2) */
34 double hipotenusa( double cateto1, double cateto2 )
35 {
36     return sqrt( cateto1 * cateto1 + cateto2 * cateto2
37 );
38 }
```

Resultado esperado: (3,4) → 5,00; (5,12) → 13,00; (8,15) → 17,00. Todas são triplas de Pitágoras (já vistas no Ex. 4.27).

Nota: usamos arrays (`double lado1[] = {...}`) – conceito que veremos formalmente no Capítulo 6. Aqui, é apenas uma forma compacta de listar os valores de teste.

9. Exercício 5.16 – Função potenciaInt

Resposta detalhada

```
1  /* Ex5.16: potencia_int.c
2     Calcula base^expoente sem usar funcoes de <math.h>
3     */
4
5  #include <stdio.h>
6
7  int potenciaInt( int base, int expoente );
8
9  int main( void )
10 {
11     int b, e;
12     printf( "Digite base e expoente positivo: " );
13     scanf( "%d%d", &b, &e );
14     printf( "%d ^ %d = %d\n", b, e, potenciaInt( b, e
15         ) );
16     return 0;
17 }
18
19 /* Calcula base elevado a expoente usando for */
20 int potenciaInt( int base, int expoente )
21 {
22     int resultado = 1;
23     int i;
24
25     for ( i = 1; i <= expoente; ++i ) {
26         resultado *= base;
27     }
28
29     return resultado;
30 }
```

Verificação: $\text{potenciaInt}(3, 4) \rightarrow 3 \times 3 \times 3 \times 3 = 81$.

10. Exercício 5.17 – Função multiple

Resposta detalhada

```
1  /* Ex5.17: multiplo.c
2     Verifica se segundo inteiro e multiplo do primeiro
3     */
4
5  #include <stdio.h>
6
7  int multiple( int a, int b );
8
9  int main( void )
10 {
11     int x, y;
12
13     printf( "Digite dois inteiros (-1 -1 para terminar
14             ): " );
15     scanf( "%d%d", &x, &y );
16
17     while ( x != -1 || y != -1 ) {
18         if ( multiple( x, y ) ) {
19             printf( "%d e multiplo de %d\n", y, x );
20         }
21         else {
22             printf( "%d NAO e multiplo de %d\n", y, x
23                     );
24         }
25         printf( "Proximos dois (-1 -1 para terminar):
26                 " );
27         scanf( "%d%d", &x, &y );
28     }
29
30     return 0;
31 }
32
33 /* Retorna 1 se b e multiplo de a, senao 0 */
34 int multiple( int a, int b )
35 {
36     if ( a == 0 ) {
37         return 0;          /* divisao por zero */
38     }
39     return ( b % a == 0 ); /* avalia para 1 ou 0 */
40 }
```

11. Exercício 5.18 – Função even

Resposta detalhada

```
1  /* Ex5.18: par.c
2     Determina se inteiro e par */
3  #include <stdio.h>
4
5  int even( int n );
6
7  int main( void )
8  {
9      int x;
10     printf( "Digite inteiro (0 para terminar): " );
11     scanf( "%d", &x );
12
13     while ( x != 0 ) {
14         if ( even( x ) ) {
15             printf( "%d e PAR\n", x );
16         }
17         else {
18             printf( "%d e IMPAR\n", x );
19         }
20         printf( "Proximo (0 para terminar): " );
21         scanf( "%d", &x );
22     }
23
24     return 0;
25 }
26
27 int even( int n )
28 {
29     return ( n % 2 == 0 ); /* 1 se par, 0 se impar */
30 }
```

12. Exercício 5.19 – Quadrado Sólido de Asteriscos

Resposta detalhada

```
1  /* Ex5.19: quadrado_asteriscos.c */
2  #include <stdio.h>
3
4  void exibeQuadrado( int lado );
5
6  int main( void )
7  {
8      int n;
9      printf( "Digite o lado: " );
```

```
10     scanf( "%d", &n );
11
12     exibeQuadrado( n );
13     return 0;
14 }
15
16 void exibeQuadrado( int lado )
17 {
18     int i, j;
19     for ( i = 1; i <= lado; ++i ) {
20         for ( j = 1; j <= lado; ++j ) {
21             printf( "*" );
22         }
23         printf( "\n" );
24     }
25 }
```

13. Exercício 5.20 – Quadrado de Caractere Variável

Resposta detalhada

```
1  /* Ex5.20: quadrado_caractere.c */
2  #include <stdio.h>
3
4  void exibeQuadrado( int lado, char fillCharacter );
5
6  int main( void )
7  {
8      int n;
9      char ch;
10
11      printf( "Digite o lado e o caractere: " );
12      scanf( "%d %c", &n, &ch ); /* espaco antes de %c
13                                  */
14
15      exibeQuadrado( n, ch );
16      return 0;
17 }
18
19 void exibeQuadrado( int lado, char fillCharacter )
20 {
21     int i, j;
22     for ( i = 1; i <= lado; ++i ) {
23         for ( j = 1; j <= lado; ++j ) {
24             printf( "%c", fillCharacter );
25         }
26         printf( "\n" );
27     }
28 }
```

```
27 }
```

14. Exercício 5.21 – Projeto: Desenhando Formas

Resposta detalhada

```
1  /* Ex5.21: formas.c
2     Programa que desenha varias formas usando funcoes
3     */
4
5  #include <stdio.h>
6
7  void quadrado      ( int lado, char ch );
8  void retangulo     ( int largura, int altura, char ch );
9  void triangulo     ( int altura, char ch );
10 void linha         ( int comprimento, char ch );
11
12 int main( void )
13 {
14     int opcao, n;
15     char ch;
16
17     printf( "Menu de formas:\n" );
18     printf( "  1 - Quadrado\n" );
19     printf( "  2 - Retangulo\n" );
20     printf( "  3 - Triangulo\n" );
21     printf( "  4 - Linha\n" );
22     printf( "Escolha: " );
23     scanf( "%d", &opcao );
24
25     switch ( opcao ) {
26     case 1:
27         printf( "Lado e caractere: " );
28         scanf( "%d %c", &n, &ch );
29         quadrado( n, ch );
30         break;
31     case 2: {
32         int alt;
33         printf( "Largura, altura, caractere: " );
34         scanf( "%d %d %c", &n, &alt, &ch );
35         retangulo( n, alt, ch );
36         break;
37     }
38     case 3:
39         printf( "Altura e caractere: " );
40         scanf( "%d %c", &n, &ch );
41         triangulo( n, ch );
42         break;
43     case 4:
44         printf( "Comprimento e caractere: " );
45         scanf( "%d %c", &n, &ch );
46         linha( n, ch );
47         break;
```

```
46         default:
47             printf( "Opcao invalida\n" );
48     }
49
50     return 0;
51 }
52
53 void quadrado( int lado, char ch )
54 {
55     int i, j;
56     for ( i = 1; i <= lado; ++i ) {
57         for ( j = 1; j <= lado; ++j ) printf( "%c", ch
58             );
59         printf( "\n" );
60     }
61 }
62
63 void retangulo( int largura, int altura, char ch )
64 {
65     int i, j;
66     for ( i = 1; i <= altura; ++i ) {
67         for ( j = 1; j <= largura; ++j ) printf( "%c",
68             ch );
69         printf( "\n" );
70     }
71 }
72
73 void triangulo( int altura, char ch )
74 {
75     int i, j;
76     for ( i = 1; i <= altura; ++i ) {
77         for ( j = 1; j <= i; ++j ) printf( "%c", ch );
78         printf( "\n" );
79     }
80 }
81
82 void linha( int comprimento, char ch )
83 {
84     int j;
85     for ( j = 1; j <= comprimento; ++j ) printf( "%c",
86         ch );
87     printf( "\n" );
88 }
```

15. Exercício 5.22 – Separando Dígitos

Resposta detalhada

```
1  /* Ex5.22: separa_digitos.c */
2  #include <stdio.h>
3
4  int parteInteira( int a, int b ); /* parte (a) */
5  int moduloInt    ( int a, int b ); /* parte (b) */
6  void exhibeDigitos( int n );      /* parte (c) */
7
8  int main( void )
9  {
10     int n;
11     printf( "Digite inteiro entre 1 e 32767: " );
12     scanf( "%d", &n );
13     exhibeDigitos( n );
14     return 0;
15 }
16
17 int parteInteira( int a, int b )
18 {
19     return a / b;
20 }
21
22 int moduloInt( int a, int b )
23 {
24     return a % b;
25 }
26
27 /* Exibe dígitos do número separados por 2 espaços */
28 void exhibeDigitos( int n )
29 {
30     int divisor = 10000; /* maior potência <= 32767
31                          */
32     int digito;
33     int comecou = 0;      /* para suprimir zeros a
34                          esquerda */
35
36     while ( divisor > 0 ) {
37         digito = parteInteira( n, divisor );
38         n = moduloInt( n, divisor );
39
40         if ( digito != 0 || comecou || divisor == 1 )
41         {
42             printf( "%d  ", digito );
43             comecou = 1;
44         }
45
46         divisor /= 10;
47     }
```

```
44     }  
45     printf( "\n" );  
46 }
```

16. Exercício 5.23 – Tempo em Segundos

Resposta detalhada

```
1  /* Ex5.23: tempo_segundos.c
2     Converte hora:minuto:segundo em segundos desde meia
3     -noite */
4
5  #include <stdio.h>
6
7  int paraSegundos( int h, int m, int s );
8
9  int main( void )
10 {
11     int h1, m1, s1, h2, m2, s2;
12     int seg1, seg2, diferenca;
13
14     printf( "Primeiro horario (h m s): " );
15     scanf( "%d%d%d", &h1, &m1, &s1 );
16
17     printf( "Segundo horario (h m s): " );
18     scanf( "%d%d%d", &h2, &m2, &s2 );
19
20     seg1 = paraSegundos( h1, m1, s1 );
21     seg2 = paraSegundos( h2, m2, s2 );
22     diferenca = seg2 - seg1;
23     if ( diferenca < 0 ) diferenca = -diferenca;
24
25     printf( "Diferenca: %d segundos\n", diferenca );
26     return 0;
27 }
28
29 int paraSegundos( int h, int m, int s )
30 {
31     return h * 3600 + m * 60 + s;
32 }
```

17. Exercício 5.24 – Conversões de Temperatura

Resposta detalhada

Fórmulas:

$$C = \frac{5}{9}(F - 32)$$
$$F = \frac{9}{5}C + 32$$

```
1  /* Ex5.24: temperaturas.c */
2  #include <stdio.h>
```

```

3
4 int celsius( int fahrenheit );
5 int fahrenheit( int celsius );
6
7 int main( void )
8 {
9     int t;
10
11     printf( "Fahrenheit -> Celsius      Celsius ->
12             Fahrenheit\n" );
13     printf( "-----
14             -----\n" );
15
16     for ( t = 32; t <= 212; t += 18 ) {
17         printf( "%4d F = %4d C      %4d C = %4d
18                 F\n",
19                 t, celsius( t ),
20                 t - 32, fahrenheit( t - 32 ) );
21     }
22
23     return 0;
24 }
25
26 int celsius( int f )
27 {
28     return ( int )( 5.0 / 9.0 * ( f - 32 ) );
29 }
30
31 int fahrenheit( int c )
32 {
33     return ( int )( 9.0 / 5.0 * c + 32 );
34 }

```

Nota: usamos *cast* para *int* para retornar o resultado truncado, conforme exigido pelo enunciado (“com inteiros”).

18. Exercício 5.25 – Mínimo de 3 Números

Resposta detalhada

```

1  /* Ex5.25: minimo_tres.c */
2  #include <stdio.h>
3
4  double menor( double a, double b, double c );
5
6  int main( void )
7  {
8      double x, y, z;
9      printf( "Digite 3 numeros: " );

```

```
10     scanf( "%lf%lf%lf", &x, &y, &z );
11
12     printf( "O menor e: %f\n", menor( x, y, z ) );
13     return 0;
14 }
15
16 double menor( double a, double b, double c )
17 {
18     double m = a;
19     if ( b < m ) m = b;
20     if ( c < m ) m = c;
21     return m;
22 }
```

Versão idiomática com operador ternário:

```
1 double menor( double a, double b, double c )
2 {
3     return ( a < b ? ( a < c ? a : c ) : ( b < c ? b :
4         c ) );
5 }
```

19. Exercício 5.26 – Números Perfeitos

Resposta detalhada

Definição: Um inteiro n é *perfeito* se a soma de seus divisores próprios (excluindo n) é igual a n .

```
1  /* Ex5.26: numeros_perfeitos.c
2     Encontra numeros perfeitos entre 1 e 1000 */
3  #include <stdio.h>
4
5  int perfect( int n );
6  void exhibeFatores( int n );
7
8  int main( void )
9  {
10     int n;
11
12     printf( "Numeros perfeitos entre 1 e 1000:\n" );
13     printf( "%-10s%s\n", "Numero", "Fatores" );
14
15     for ( n = 2; n <= 1000; ++n ) {
16         if ( perfect( n ) ) {
17             printf( "%-10d", n );
18             exhibeFatores( n );
19             printf( "\n" );
20         }
21     }
22
23     return 0;
24 }
25
26 /* Retorna 1 se n e perfeito, senao 0 */
27 int perfect( int n )
28 {
29     int soma = 1;          /* 1 e sempre fator (
30                             excluindo n) */
31     int i;
32
33     for ( i = 2; i * i <= n; ++i ) {
34         if ( n % i == 0 ) {
35             soma += i;
36             if ( i != n / i ) { /* evita contar raiz
37                                 quadrada 2x */
38                 soma += n / i;
39             }
40         }
41     }
42
43     return ( n > 1 && soma == n );
44 }
```

```

43
44 void exhibeFatores( int n )
45 {
46     int i;
47     printf( "1 " );
48     for ( i = 2; i <= n / 2; ++i ) {
49         if ( n % i == 0 ) printf( "%d ", i );
50     }
51 }

```

Resultado: Os primeiros números perfeitos são **6** ($1 + 2 + 3$), **28** ($1 + 2 + 4 + 7 + 14$), **496**, e **8128**.

20. Exercício 5.27 – Números Primos

Resposta detalhada

```

1  /* Ex5.27: primos.c */
2  #include <stdio.h>
3  #include <math.h>
4
5  int primo( int n );
6
7  int main( void )
8  {
9      int n;
10     int contador = 0;
11
12     printf( "Primos de 1 a 10000:\n" );
13
14     for ( n = 2; n <= 10000; ++n ) {
15         if ( primo( n ) ) {
16             ++contador;
17             printf( "%6d", n );
18             if ( contador % 10 == 0 ) printf( "\n" );
19         }
20     }
21
22     printf( "\n\nTotal de primos: %d\n", contador );
23     return 0;
24 }
25
26 /* Retorna 1 se n e primo, senao 0
27    So precisa testar ate sqrt(n) -- ver explicacao
28    abaixo */
29 int primo( int n )
30 {
31     int i;
32     int limite = ( int )sqrt( ( double )n );

```

```
32
33     if ( n < 2 ) return 0;
34     if ( n == 2 ) return 1;
35     if ( n % 2 == 0 ) return 0;  /* pares > 2 nao sao
                                   primos */
36
37     for ( i = 3; i <= limite; i += 2 ) {
38         if ( n % i == 0 ) return 0;
39     }
40
41     return 1;
42 }
```

Por que apenas até $\sqrt{n}$? Se $n = a \times b$ e ambos $a, b > \sqrt{n}$, então $a \times b > n$, contradição. Portanto, ao menos um dos fatores deve ser $\leq \sqrt{n}$. Se não encontrarmos divisor até $\sqrt{n}$, n é primo.

Ganho de desempenho: para $n = 10\,000$, $\sqrt{n} = 100$. Em vez de testar 5 000 valores ($n/2$), testamos apenas 100 – redução de 50×!

21. Exercício 5.28 – Inverter Dígitos

Resposta detalhada

```
1  /* Ex5.28: inverte_digitos.c */
2  #include <stdio.h>
3
4  int inverter( int n );
5
6  int main( void )
7  {
8      int n;
9      printf( "Digite um inteiro: " );
10     scanf( "%d", &n );
11     printf( "Invertido: %d\n", inverter( n ) );
12     return 0;
13 }
14
15 int inverter( int n )
16 {
17     int invertido = 0;
18     while ( n > 0 ) {
19         invertido = invertido * 10 + n % 10;
20         n /= 10;
21     }
22     return invertido;
23 }
```

Trace para 7631: $i = 0, n = 7631 \rightarrow i = 1, n = 763 \rightarrow i = 13, n = 76 \rightarrow i = 136, n = 7 \rightarrow i = 1367, n = 0$. Resultado: **1367**.

22. Exercício 5.29 – MDC (Algoritmo de Euclides)

Resposta detalhada

Algoritmo de Euclides: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ até que $b = 0$.

```
1  /* Ex5.29: mdc.c */
2  #include <stdio.h>
3
4  int mdc( int a, int b );
5
6  int main( void )
7  {
8      int x, y;
9      printf( "Digite 2 inteiros positivos: " );
10     scanf( "%d%d", &x, &y );
11     printf( "MDC(%d, %d) = %d\n", x, y, mdc( x, y ) );
12     return 0;
13 }
```

```
14
15 int mdc( int a, int b )
16 {
17     int r;
18     while ( b != 0 ) {
19         r = a % b;
20         a = b;
21         b = r;
22     }
23     return a;
24 }
```

Exemplo: $\text{gcd}(48, 36) = \text{gcd}(36, 12) = \text{gcd}(12, 0) = 12$.

23. Exercício 5.30 – Pontos de Qualidade

Resposta detalhada

```
1  /* Ex5.30: quality_points.c */
2  #include <stdio.h>
3
4  int qualityPoints( int media );
5
6  int main( void )
7  {
8      int m;
9      printf( "Digite a media (0-100): " );
10     scanf( "%d", &m );
11     printf( "Pontos de qualidade: %d\n", qualityPoints
12           ( m ) );
13     return 0;
14 }
15
16 int qualityPoints( int media )
17 {
18     if ( media >= 90 ) return 4;
19     if ( media >= 80 ) return 3;
20     if ( media >= 70 ) return 2;
21     if ( media >= 60 ) return 1;
22     return 0;
23 }
```

24. Exercício 5.31 – Lançamento de Moeda

Resposta detalhada

```
1  /* Ex5.31: moeda.c
2     Simula 100 lançamentos de moeda */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <time.h>
6
7  int flip( void );
8
9  int main( void )
10 {
11     int caras = 0;
12     int coroas = 0;
13     int i;
14
15     srand( time( NULL ) ); /* randomiza */
16
17     for ( i = 1; i <= 100; ++i ) {
18         if ( flip() == 0 ) {
19             ++caras;
20             printf( "Cara " );
21         }
22         else {
23             ++coroas;
24             printf( "Coroa " );
25         }
26         if ( i % 5 == 0 ) printf( "\n" );
27     }
28
29     printf( "\nTotal: %d caras, %d coroas\n", caras,
30           coroas );
31     return 0;
32 }
33
34 int flip( void )
35 {
36     return rand() % 2; /* 0 ou 1 com igual
37                        probabilidade */
38 }
```

Resultado esperado: aproximadamente 50 caras e 50 coroas (variação típica de ± 5 a ± 10 pela natureza pseudoaleatória).

25. Exercício 5.32 – Descubra o Número

Resposta detalhada

```
1  /* Ex5.32: descubra_numero.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5
6  int main( void )
7  {
8      int numeroSecreto;
9      int tentativa;
10     char resposta;
11
12     srand( time( NULL ) );
13
14     do {
15         numeroSecreto = 1 + rand() % 1000;
16
17         printf( "Eu tenho um numero entre 1 e 1000.\n"
18             );
19         printf( "Voce consegue descobrir qual e?\n" );
20         printf( "Digite sua primeira tentativa: " );
21         scanf( "%d", &tentativa );
22
23         while ( tentativa != numeroSecreto ) {
24             if ( tentativa < numeroSecreto ) {
25                 printf( "Muito baixo. Tente novamente
26                     .\n" );
27             }
28             else {
29                 printf( "Muito alto. Tente novamente.\n
30                     " );
31             }
32             scanf( "%d", &tentativa );
33         }
34
35         printf( "Excelente! Voce descobriu o numero!\n
36             " );
37         printf( "Gostaria de jogar novamente (s ou n)?
38             " );
39         scanf( " %c", &resposta );
40
41     } while ( resposta == 's' || resposta == 'S' );
42
43     return 0;
44 }
```

26. Exercício 5.33 – Modificação de “Descubra o Número”

Resposta detalhada

Por que 10 ou menos tentativas bastam?

A técnica usada é **busca binária**: a cada tentativa, dividimos o intervalo ao meio. Como $2^{10} = 1024 > 1000$, em **no máximo 10** divisões pelo meio consegue-se localizar qualquer número de 1 a 1000.

Estratégia ótima: chute sempre o meio do intervalo restante. Começa com 500. Se “muito alto”, chute 250. Se “muito baixo”, chute 125. E assim por diante.

```
1  /* Ex5.33: modificacao do jogo */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5
6  int main( void )
7  {
8      int numeroSecreto, tentativa, contador;
9      char resposta;
10
11     srand( time( NULL ) );
12
13     do {
14         numeroSecreto = 1 + rand() % 1000;
15         contador = 0;
16
17         printf( "Tente descobrir o numero (1 a 1000):
18                 " );
19
20         do {
21             scanf( "%d", &tentativa );
22             ++contador;
23
24             if ( tentativa < numeroSecreto ) {
25                 printf( "Muito baixo. Tente novamente:
26                         " );
27             }
28             else if ( tentativa > numeroSecreto ) {
29                 printf( "Muito alto. Tente novamente:
30                         " );
31             }
32         } while ( tentativa != numeroSecreto );
33
34         printf( "Excelente! Acertou em %d tentativas.\n", contador );
35
36         if ( contador < 10 ) {
37             printf( "Ou voce conhece o segredo ou teve
38                     sorte!\n" );
39         }
40     }
```

```
35         }
36         else if ( contador == 10 ) {
37             printf( "Ahah! Voce conhece o segredo!\n"
38                 );
39         }
40         else {
41             printf( "Voce deveria se sair melhor!\n" )
42                 ;
43         }
44
45         printf( "Jogar novamente? (s/n): " );
46         scanf( " %c", &resposta );
47     } while ( resposta == 's' || resposta == 'S' );
48
49     return 0;
50 }
```

27. Exercício 5.34 – Exponenciação Recursiva

Resposta detalhada

Definição recursiva: $\text{base}^{\text{exp}} = \text{base} \times \text{base}^{\text{exp}-1}$, com caso básico $\text{base}^1 = \text{base}$.

```
1  /* Ex5.34: power_recursiva.c */
2  #include <stdio.h>
3
4  int power( int base, int expoente );
5
6  int main( void )
7  {
8      int b, e;
9      printf( "Digite base e expoente: " );
10     scanf( "%d%d", &b, &e );
11     printf( "%d ^ %d = %d\n", b, e, power( b, e ) );
12     return 0;
13 }
14
15 int power( int base, int expoente )
16 {
17     if ( expoente == 1 ) {           /* caso basico */
18         return base;
19     }
20     return base * power( base, expoente - 1 ); /*
21                                     recursao */
22 }
```

Trace para $\text{power}(3, 4)$:

- $\text{power}(3, 4) = 3 \times \text{power}(3, 3)$
- $\text{power}(3, 3) = 3 \times \text{power}(3, 2)$
- $\text{power}(3, 2) = 3 \times \text{power}(3, 1)$
- $\text{power}(3, 1) = 3$ (caso básico)

Resultado: $3 \times 3 \times 3 \times 3 = 81$.

28. Exercício 5.35 – Fibonacci Iterativo com double

Resposta detalhada

Versão não recursiva:

```
1  /* Ex5.35: fibonacci_iterativo.c */
2  #include <stdio.h>
3
4  double fibonacci( int n );
5
6  int main( void )
7  {
```

```
8      int n = 0;
9
10     /* Loop ate o resultado falhar (overflow) */
11     while ( 1 ) {
12         printf( "fib(%d) = %.0f\n", n, fibonacci( n )
13             );
14         ++n;
15     }
16     return 0;
17 }
18
19 double fibonacci( int n )
20 {
21     double anterior = 0.0;
22     double atual = 1.0;
23     double proximo;
24     int i;
25
26     if ( n == 0 ) return 0.0;
27     if ( n == 1 ) return 1.0;
28
29     for ( i = 2; i <= n; ++i ) {
30         proximo = anterior + atual;
31         anterior = atual;
32         atual = proximo;
33     }
34
35     return atual;
36 }
```

Limites: double representa até cerca de $1,8 \times 10^{308}$. $F_{1476} \approx 1,3 \times 10^{308}$, então em torno desse índice começam os *overflows* (resultando em *inf*).

29. Exercício 5.36 – Torres de Hanói

Resposta detalhada

Decomposição recursiva: Para mover n discos do pino A para o pino C usando B como auxiliar:

- Mover $n - 1$ discos de A para B (usando C como auxiliar).
- Mover o maior disco de A para C.
- Mover $n - 1$ discos de B para C (usando A como auxiliar).

```
1  /* Ex5.36: torres_hanoi.c */
2  #include <stdio.h>
3
4  void hanoi( int n, int origem, int destino, int
5             auxiliar );
6
7  int main( void )
8  {
9      int n;
10     printf( "Numero de discos: " );
11     scanf( "%d", &n );
12
13     printf( "Sequencia de movimentos:\n" );
14     hanoi( n, 1, 3, 2 ); /* origem=1, destino=3,
15                           auxiliar=2 */
16
17     return 0;
18 }
19
20 void hanoi( int n, int origem, int destino, int
21            auxiliar )
22 {
23     if ( n == 1 ) {
24         /* caso basico */
25         printf( "%d -> %d\n", origem, destino );
26         return;
27     }
28     /* Move n-1 discos de origem para auxiliar */
29     hanoi( n - 1, origem, auxiliar, destino );
30
31     /* Move o disco maior */
32     printf( "%d -> %d\n", origem, destino );
33
34     /* Move n-1 discos de auxiliar para destino */
35     hanoi( n - 1, auxiliar, destino, origem );
36 }
```

Saída para $n=3$:

Saída do programa

```

1 -> 3
1 -> 2
3 -> 2
1 -> 3
2 -> 1
2 -> 3
1 -> 3

```

Curiosidade: para n discos são necessários $2^n - 1$ movimentos. Para os 64 discos da lenda, $2^{64} - 1 \approx 1,8 \times 10^{19}$ movimentos – mesmo a 1 movimento por segundo, levaria ~ 585 bilhões de anos!

30. Exercício 5.38 – Visualizando a Recursão (Fatorial)

Resposta detalhada

```

1  /* Ex5.38: fatorial_visual.c */
2  #include <stdio.h>
3
4  long fatorial( int n, int nivel );
5
6  int main( void )
7  {
8      int n;
9      printf( "Digite n: " );
10     scanf( "%d", &n );
11     printf( "Resultado: %ld\n", fatorial( n, 0 ) );
12     return 0;
13 }
14
15 long fatorial( int n, int nivel )
16 {
17     int i;
18     long resultado;
19
20     /* Recuo proporcional ao nivel */
21     for ( i = 0; i < nivel; ++i ) printf( "  " );
22     printf( "fatorial(%d) chamado\n", n );
23
24     if ( n <= 1 ) {
25         for ( i = 0; i < nivel; ++i ) printf( "  " );
26         printf( "fatorial(%d) retorna 1\n", n );
27         return 1;
28     }
29
30     resultado = n * fatorial( n - 1, nivel + 1 );
31

```

```
32     for ( i = 0; i < nivel; ++i ) printf( "  " );
33     printf( "fatorial(%d) retorna %ld\n", n, resultado
34           );
35     return resultado;
36 }
```

Saída para n=4:

Saída do programa

```
fatorial(4) chamado
  fatorial(3) chamado
    fatorial(2) chamado
      fatorial(1) chamado
      fatorial(1) retorna 1
    fatorial(2) retorna 2
  fatorial(3) retorna 6
fatorial(4) retorna 24
```

31. Exercício 5.39 – MDC Recursivo

Resposta detalhada

```
1  /* Ex5.39: mdc_recursivo.c */
2  #include <stdio.h>
3
4  int mdc( int x, int y );
5
6  int main( void )
7  {
8      int a, b;
9      printf( "Digite 2 inteiros: " );
10     scanf( "%d%d", &a, &b );
11     printf( "MDC(%d, %d) = %d\n", a, b, mdc( a, b ) );
12     return 0;
13 }
14
15 int mdc( int x, int y )
16 {
17     if ( y == 0 ) return x;           /* caso basico */
18     return mdc( y, x % y );         /* recursao */
19 }
```

Trace para `mdc(48, 36)`:

- `mdc(48, 36) → mdc(36, 48 % 36) = mdc(36, 12)`
- `mdc(36, 12) → mdc(12, 0)`
- `mdc(12, 0) → retorna 12 (caso básico)`

Resultado: **12**.

32. Exercício 5.40 – main Recursiva

Resposta detalhada

```
1  /* Ex5.40: main_recursiva.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      static int count = 1;
7      printf( "Chamada #%d\n", count );
8      count++;
9      main();           /* chama-se recursivamente */
10     return 0;
11 }
```

O que acontece? A função `main` **pode** chamar a si mesma em C (não há restrição da linguagem para isso). O programa imprime sucessivamente: Chamada #1, Chamada #2, Chamada #3...até que a **pilha de execução**

transborde (*stack overflow*). Cada chamada consome espaço de pilha; eventualmente, o sistema operacional encerra o programa com falha de segmentação.

Nota: embora compile e execute, é uma prática **altamente desencorajada**. A recursão deve ter um caso básico que termine; aqui não há.

33. Exercício 5.41 – Distância entre Pontos

Resposta detalhada

```
1  /* Ex5.41: distancia.c */
2  #include <stdio.h>
3  #include <math.h>
4
5  double distance( double x1, double y1, double x2,
6                  double y2 );
7
8  int main( void )
9  {
10     double x1, y1, x2, y2;
11     printf( "Digite o primeiro ponto (x1 y1): " );
12     scanf( "%lf%lf", &x1, &y1 );
13     printf( "Digite o segundo ponto (x2 y2): " );
14     scanf( "%lf%lf", &x2, &y2 );
15     printf( "Distancia: %.4f\n",
16            distance( x1, y1, x2, y2 ) );
17     return 0;
18 }
19
20 double distance( double x1, double y1, double x2,
21                 double y2 )
22 {
23     double dx = x2 - x1;
24     double dy = y2 - y1;
25     return sqrt( dx * dx + dy * dy );
26 }
```

34. Exercício 5.42 – O que o programa faz?

Resposta detalhada

```
1  if ( ( c = getchar() ) != EOF ) {  
2      main();  
3      printf( "%c", c );  
4  }
```

Análise: O programa lê caracteres do teclado um a um e usa recursão através de `main()`. Para cada caractere lido, antes de imprimi-lo, chama `main()` recursivamente para ler e processar o próximo. Quando EOF é encontrado, a recursão começa a desempilhar – imprimindo os caracteres **na ordem inversa**.

Conclusão: o programa **inverte uma sequência de caracteres da entrada**.

Exemplo: entrada ABC<EOF> produz saída CBA.

35. Exercício 5.43 – O que faz mystery?

Resposta detalhada

```
1  int mystery( int a, int b )  
2  {  
3      if ( b == 1 ) return a;  
4      else return a + mystery( a, b - 1 );  
5  }
```

Análise: Para `mystery(a, b)`:

- Se $b = 1$: retorna a .
- Senão: retorna $a + \text{mystery}(a, b - 1)$.

Expandindo: $\text{mystery}(a, b) = a + a + \dots + a$ (b vezes) $= a \times b$.

Conclusão: a função calcula a **multiplicação $a \times b$ por adições sucessivas** – uma forma recursiva clássica de implementar multiplicação.

Restrição: b deve ser positivo para evitar recursão infinita.

36. Exercício 5.44 – mystery sem restrição em b

Resposta detalhada

Para tratar b negativo ou zero:

```
1  int mystery( int a, int b )  
2  {  
3      if ( b == 0 ) return 0;           /* a * 0 = 0 */  
4      if ( b < 0 ) {  
5          return -mystery( a, -b );    /* a * (-b) = -(a *  
                                         b) */
```

```
6     }  
7     if ( b == 1 ) return a;  
8     return a + mystery( a, b - 1 );  
9 }
```

Lógica:

- Se $b = 0$, o produto é 0.
- Se $b < 0$, calculamos com $|b|$ e negamos o resultado.
- Caso contrário, mantemos a recursão original.

37. Exercício 5.46 – Identificar e Corrigir Erros

Resposta detalhada

- a) **Erro:** O cabeçalho da definição não tem o tipo de retorno – deveria ser `double cube(float number)`.

Correção:

```
1 double cube( float ); /* prototipo */
2 double cube( float number ) /* definicao */
3 {
4     return number * number * number;
5 }
```

- b) **Erro:** Combinar dois especificadores de classe de armazenamento (`register` e `auto`) na mesma declaração é proibido.

Correção: usar apenas um:

```
1 register int x = 7; /* OU: auto int x = 7; */
```

- c) **Erro:** `srand` é uma função `void` (não retorna valor) e exige um argumento. Não pode ser usada em uma atribuição como aqui.

Correção: provavelmente o autor quis usar `rand`:

```
1 int randomNumber = rand(); /* obtem um aleatorio
   */
2 srand( time( NULL ) );      /* uso correto de srand
   */
```

- d) **Erro:** A conversão (`double`) `x` no `printf` é redundante e fora de propósito. O verdadeiro problema é exibir um `int` com `%f`, o que é incorreto.

Correção:

```
1 double y = 123.45678;
2 int x;
3 x = (int) y;          /* cast explícito recomendado */
4 printf( "%d\n", x );
```

- e) **Erro:** A variável `number` é *redeclarada* dentro da função, sobrepondo o parâmetro do mesmo nome (Erro Comum 5.5).

Correção: remover a redeclaração:

```
1 double square( double number )
2 {
3     return number * number; /* sem 'double number
   ;' aqui */
4 }
```

- f) **Erro:** Na recursão, a chamada `sum(n)` usa o mesmo `n`, sem reduzir – gera *recursão infinita*.

Correção:

```
1 int sum( int n )
```

```
2 {  
3     if ( n == 0 ) return 0;  
4     return n + sum( n - 1 );  /* deve reduzir o  
                               argumento */  
5 }
```

38. Exercício 5.47 – Modificação do Jogo Craps

Resposta detalhada

```
1  /* Ex5.47: craps_apostas.c  
2     Versao do jogo craps com sistema de apostas */  
3  #include <stdio.h>  
4  #include <stdlib.h>  
5  #include <time.h>  
6  
7  int rolarDados( void );  
8  
9  enum Status { CONTINUA, VENCEU, PERDEU };  
10  
11 int main( void )  
12 {  
13     int saldoBanca = 1000;  
14     int aposta;  
15     int soma, meuPonto;  
16     enum Status status;  
17  
18     srand( time( NULL ) );  
19  
20     while ( saldoBanca > 0 ) {  
21         printf( "\nSaldo: R$ %d\n", saldoBanca );  
22         printf( "Aposta (max %d): ", saldoBanca );  
23         scanf( "%d", &aposta );  
24  
25         while ( aposta <= 0 || aposta > saldoBanca ) {  
26             printf( "Aposta invalida! Digite (max %d):  
27                 ",  
28                     saldoBanca );  
29             scanf( "%d", &aposta );  
30         }  
31  
32         soma = rolarDados();  
33         switch ( soma ) {  
34             case 7:  
35             case 11:  
36                 status = VENCEU;  
37                 break;  
38             case 2:
```

```
38         case 3:
39         case 12:
40             status = PERDEU;
41             break;
42         default:
43             status = CONTINUA;
44             meuPonto = soma;
45             printf( "Ponto e %d\n", meuPonto );
46             break;
47     }
48
49     while ( status == CONTINUA ) {
50         soma = rolarDados();
51         if ( soma == meuPonto ) status = VENCEU;
52         else if ( soma == 7 ) status = PERDEU;
53     }
54
55     if ( status == VENCEU ) {
56         saldoBanca += aposta;
57         printf( "Voce venceu! Novo saldo: R$ %d\n"
58             , saldoBanca );
59         printf( "Voce esta ganhando. Aproveite
60             suas fichas!\n" );
61     }
62     else {
63         saldoBanca -= aposta;
64         printf( "Voce perdeu. Novo saldo: R$ %d\n"
65             , saldoBanca );
66         if ( saldoBanca == 0 ) {
67             printf( "Sinto muito. Voce esta
68                 quebrado!\n" );
69             break;
70         }
71         printf( "Vamos la, continue tentando!\n" )
72         ;
73     }
74 }
75
76 return 0;
77
78 int rolarDados( void )
79 {
80     int dado1 = 1 + rand() % 6;
81     int dado2 = 1 + rand() % 6;
82     int soma = dado1 + dado2;
83     printf( "Dados: %d + %d = %d\n", dado1, dado2,
84         soma );
85     return soma;
86 }
```


39. Exercício 5.48 – Aperfeiçoando Fibonacci Recursivo

Resposta detalhada

O problema da implementação ingênua:

A versão recursiva clássica:

```
1 long fib( int n ) {  
2     if ( n <= 1 ) return n;  
3     return fib( n - 1 ) + fib( n - 2 );  
4 }
```

tem complexidade **exponencial** ($O(\phi^n)$, onde $\phi \approx 1,618$), porque *recalcula* os mesmos valores muitas vezes. Para `fib(40)`, são feitas $\approx 10^9$ chamadas!

Técnicas de melhoria:

1. Versão iterativa (Ex. 5.35):

```
1 long fib( int n ) {  
2     long a = 0, b = 1, c;  
3     int i;  
4     for ( i = 2; i <= n; ++i ) {  
5         c = a + b;  
6         a = b;  
7         b = c;  
8     }  
9     return ( n == 0 ? 0 : b );  
10 }
```

Complexidade $O(n)$, espaço $O(1)$.

2. Recursão de cauda (*tail recursion*):

```
1 long fibAux( int n, long a, long b ) {  
2     if ( n == 0 ) return a;  
3     return fibAux( n - 1, b, a + b );  
4 }  
5 long fib( int n ) { return fibAux( n, 0, 1 ); }
```

Compiladores podem otimizar (eliminar a chamada da pilha), tornando praticamente equivalente à versão iterativa.

3. Memoização: armazenar resultados em uma tabela para evitar recálculos. Reduz a complexidade da versão recursiva de $O(\phi^n)$ para $O(n)$, mas com custo de memória $O(n)$.

Comparação dos méritos:

- **Iterativa:** mais rápida, menos memória, mais difícil de ler para problemas naturalmente recursivos.
- **Tail recursion:** elegância da recursão + eficiência da iteração (quando o compilador otimiza).
- **Memoização:** preserva clareza recursiva, mas exige memória adicional. Excelente para problemas com sobreposição de subproblemas.